

SYSTEM DESIGN INTERVIEW

# Design Spam Detection

real-time spam and abuse classification at planet scale (email, comments, posts, listings)

~ 45 minute read

© Study Guide — For personal use only

## What's Inside

---

- 01 · Problem Statement
- 02 · Requirements
- 03 · Capacity Estimation
- 04 · High-Level Architecture
- 05 · Layered Defence
- 06 · Sender Reputation
- 07 · Content Fingerprinting
- 08 · Feature Extraction
- 09 · Model Stack
- 10 · Decision & Audit Log
- 11 · Feedback Loop

- 12** · Adversarial ML
- 13** · Scalability & Distribution
- 14** · Failure Modes & Recovery
- 15** · Design Trade-offs
- 16** · Interview Playbook

# 01

## Problem Statement

Requirements and scope

Design a **spam and abuse detection** system that scores every incoming message in real time and decides whether it lands in the inbox, the spam folder, or is dropped outright. The same pipeline must extend to comments, social-media posts and marketplace listings. The threat model is **adversarial**: spammers actively probe and mutate to evade detection.

### Clarifying Questions

| Question        | Answer                                                          |
|-----------------|-----------------------------------------------------------------|
| Surfaces?       | Email primarily; reuse pipeline for comments / posts / listings |
| Volume?         | ~300 B emails/day globally; 3.5 M/sec at peak                   |
| Latency budget? | Inline scoring p99 < 50 ms (must not delay delivery)            |
| Action set?     | ALLOW / SPAM-FOLDER / QUARANTINE / DROP / CHALLENGE             |
| Feedback?       | Users mark 'Report spam' / 'Not spam'; bounces; engagement      |
| Adversarial?    | Yes — content mutation, image-only spam, URL shorteners         |
| Privacy?        | Body never leaves region; features hashed before storage        |
| Multi-tenant?   | Yes, per-tenant rules + global model                            |

## 02

## Requirements

Functional and non-functional

### Functional Requirements

| Requirement         | Details                                                            |
|---------------------|--------------------------------------------------------------------|
| Inline scoring      | Synchronous score on every message; verdict before delivery        |
| Layered defence     | Pre-filters → rules → ML classifier → deep model (borderline only) |
| Sender reputation   | Per-IP, per-domain, per-ASN scores updated continuously            |
| Content fingerprint | SimHash + LSH for near-duplicate spam waves                        |
| Feedback loop       | Capture user reports; nightly retrain; champion / challenger       |
| Audit & explain     | Persist verdict + features for every message (90-day TTL)          |
| Appeals             | Users / senders can request review of decisions                    |

### Non-Functional Requirements

| Requirement     | Target                                                      |
|-----------------|-------------------------------------------------------------|
| Throughput      | 3.5 M messages/sec at peak; 300 B/day                       |
| Latency         | Pre-filter p99 < 5 ms; full pipeline p99 < 50 ms            |
| Availability    | 99.99% — fail-open to 'ALLOW with low confidence' on outage |
| False positive  | < 0.1% (legit mail in spam folder is the worst error)       |
| False negative  | < 1% catch rate floor on bulk spam                          |
| Model freshness | Hot blocklists in < 60 s; full model nightly                |
| Storage         | ~ 1.5 PB raw events / day; 90-day verdict log               |

## 03

# Capacity Estimation

Math for planet-scale email

## Traffic

- Global email: **~300 B/day** (industry estimate, all senders)
- Average rate:  $300 \text{ B} / 86,400 \text{ s} \approx \mathbf{3.47 \text{ M/sec}}$
- Peak factor  $1.0\times$  (always-on traffic)  $\rightarrow$  call it **3.5 M/sec at peak**
- Comments / posts / listings reuse the same path at  $\sim 10\%$  of email volume
- **~95% of mail is rejected before bytes hit the model** (RBL, SPF, rate)

## Layered Funnel

| Stage                              | QPS in    | QPS out   | Reject rate                           |
|------------------------------------|-----------|-----------|---------------------------------------|
| L0 IP / RBL pre-filter             | 3.5 M/sec | 350 K/sec | $\sim 90\%$                           |
| L1 SPF/DKIM/DMARC + reputation     | 350 K/sec | 70 K/sec  | $\sim 80\%$                           |
| L2 Hand-crafted rules              | 70 K/sec  | 30 K/sec  | $\sim 60\%$                           |
| L3 GBDT classifier (LightGBM)      | 30 K/sec  | 3 K/sec   | $\sim 90\%$ scored, $10\%$ borderline |
| L4 Deep model (BERT) on borderline | 3 K/sec   | n/a       | final adjudication                    |

## Storage

- Verdict log:  $300 \text{ B/day} \times 500 \text{ B per row} \approx \mathbf{150 \text{ TB/day raw}} \rightarrow \sim \mathbf{50 \text{ TB compressed}}$
- 90-day retention:  $90 \times 50 \text{ TB} \approx \mathbf{4.5 \text{ PB}}$  compressed (Parquet on S3)
- Sender reputation hot store:  $100 \text{ M IPs} \times 200 \text{ B} \approx \mathbf{20 \text{ GB Redis}}$
- Sender reputation warm:  $5 \text{ B (IP, domain, ASN)} \times 200 \text{ B} \approx \mathbf{1 \text{ TB RocksDB}}$
- Fingerprint index (SimHash):  $30 \text{ d} \times 1 \text{ B fingerprints} \times 24 \text{ B} \approx \mathbf{720 \text{ GB}}$

## Compute

- L3 LightGBM @ 30 K/sec, 1 ms / score  $\rightarrow \sim 30 \text{ cores}$ . Replicate  $5\times$  for HA  $\rightarrow \mathbf{150 \text{ cores}}$
- L4 BERT-small @ 3 K/sec, 8 ms on T4  $\rightarrow \sim 24 \text{ GPU-seconds/sec} \rightarrow \sim \mathbf{30 \text{ T4 GPUs}}$
- Feature store reads:  $\sim 200 \text{ K/sec}$  (10 features  $\times$  L1 funnel)  $\rightarrow$  5-node Redis cluster
- Training: 30 days of labels  $\times$  1 B labels/day  $\approx \mathbf{30 \text{ B examples}}$ ; nightly Spark/Ray job

## KEY

### Key Numbers

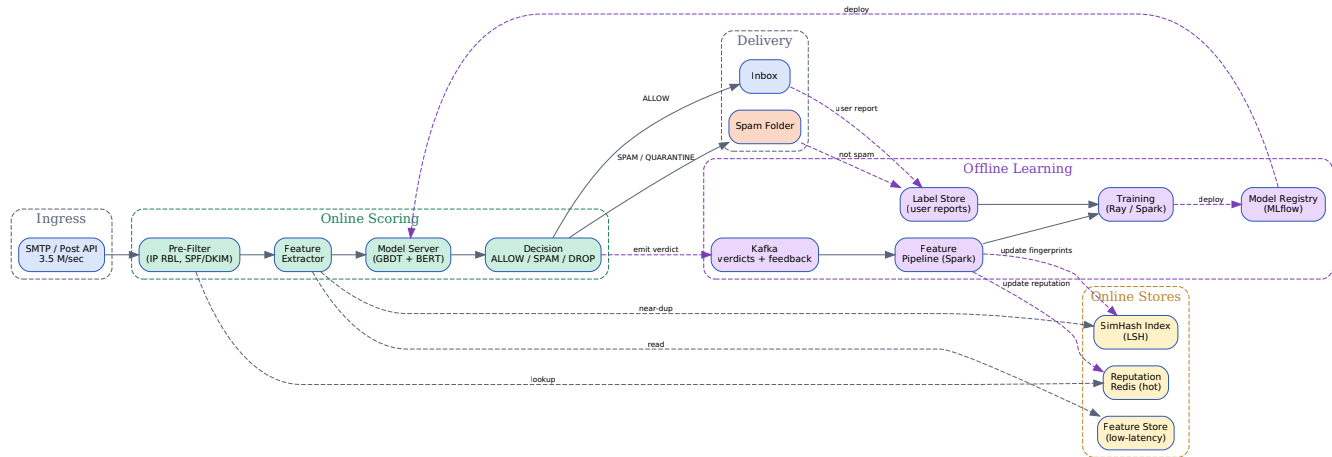
**3.5 M/sec** peak · **p99 < 50 ms** · **FP < 0.1%** / FN < 1% · **5 layers**, each rejects ~10× · **~30 GPUs** for borderline BERT · **4.5 PB** verdict log / 90 d.

# 04

## High-Level Architecture

Online scoring + offline learning

Two loops cooperate. The **online (synchronous) path** sits in the SMTP / posting hot path: pre-filter, feature extractor, model server, decision. The **offline (training) path** tails verdict events and user feedback into Kafka, builds features, trains models, and pushes new artefacts back into the model server through a model registry.



Online scoring path (top, blue) and offline learning loop (bottom, purple). The model server reads features from the online feature store; the training pipeline tails Kafka and writes back to the registry.

### Component Responsibilities

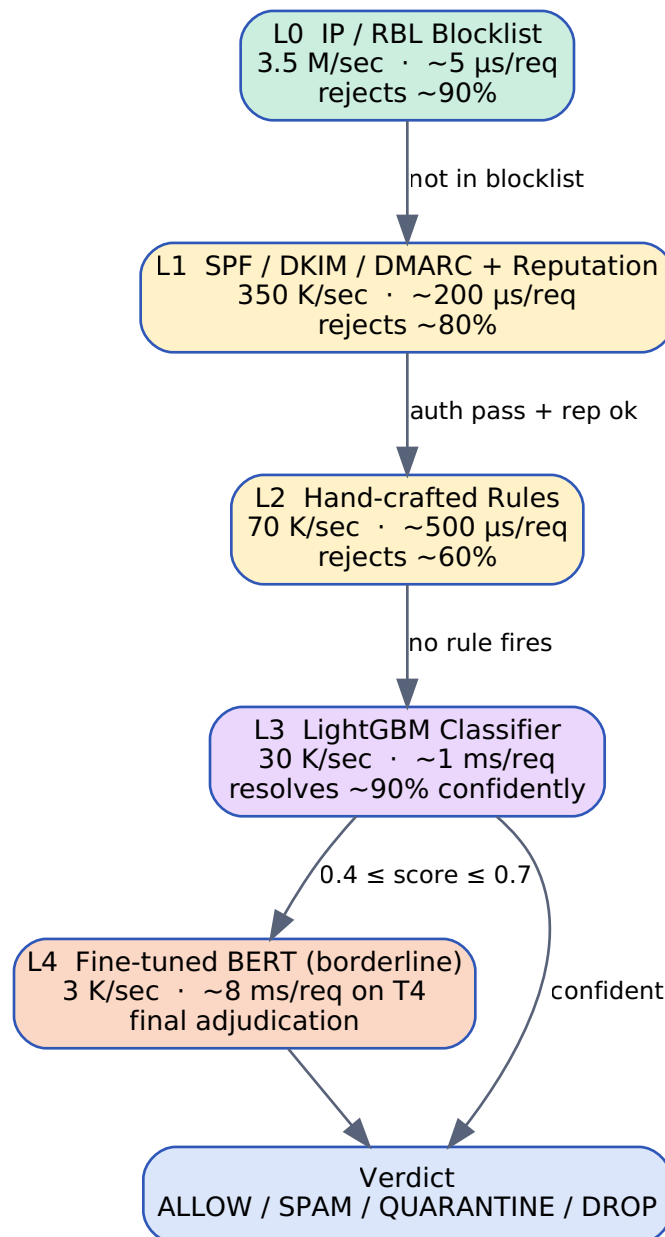
- **Pre-Filter:** O( $\mu$ s) lookups in IP / domain / ASN blocklists; SPF / DKIM / DMARC verification; rate limits
- **Feature Extractor:** tokenises body, computes SimHash, joins with reputation + recent-history features
- **Model Server:** shadow-aware ensemble; GBDT for routine traffic, BERT only for borderline scores
- **Decision:** applies thresholds; produces audit row; emits Kafka event
- **Feature Pipeline:** hourly (warm) and nightly (cold) jobs that update reputation + label store
- **Training:** nightly retrain using last-30-day labels; canary in shadow mode before promotion

# 05

## Layered Defence

Why five layers beat one big model

A single neural network at 3.5 M/sec is operationally and economically untenable. Stacking **cheap** → **expensive** filters lets each layer reject ~10× of remaining traffic, so the heaviest model only sees 0.1% of inputs. Each layer has different blast radius, latency and false-positive cost.



Volume halves by an order of magnitude per layer. Cost-per-evaluation rises in the opposite direction:  $O(1)$  Redis lookup at L0,  $O(N)$  BERT inference at L4.



## Why Each Layer Earns Its Keep

---

- **L0 IP / RBL:** 90% of spam comes from a *tiny* set of compromised hosts. Catching them in a 5  $\mu$ s Redis call frees the entire fleet.
- **L1 Auth + reputation:** SPF/DKIM/DMARC + per-domain history shields against header spoofing; cheap, deterministic, explainable
- **L2 Rules:** human-readable, fast to ship for an emerging campaign (hours vs. nightly retrain). Trade specificity for recall
- **L3 LightGBM:** dense numerical + categorical features (reputation, time-of-day, link-count). Sub-ms latency, well-calibrated
- **L4 BERT:** only borderline cases (~10% of L3 traffic). Reads body text; expensive but high precision on novel phishing

### INSIGHT

#### Cost-per-decision discipline

If a layer cannot reject  $\sim 10\times$  of remaining traffic at less than  $10\times$  the cost of the previous layer, fold it back into the previous layer. The five-layer cascade only works when each step pulls its weight.

## 06

## Sender Reputation

Per-IP / per-domain / per-ASN scores

Reputation is the single highest-signal feature. We track it at three granularities so a campaign cannot trivially hop IPs to escape: **IP, domain, ASN**. Hot lookups land in Redis; the warm tier lives in RocksDB, fed by the offline feature pipeline.

### Reputation Schema

```
-- Warm tier (RocksDB key-value, key = entity_type:entity_id)
TABLE sender_reputation (
  entity_type  ENUM('IP', 'DOMAIN', 'ASN'),
  entity_id    VARCHAR(64),          -- IPv6 or FQDN or AS#####
  msg_total_30d BIGINT,
  msg_spam_30d  BIGINT,
  spam_rate_30d FLOAT,              -- spam_30d / total_30d
  user_reports  BIGINT,
  bounce_rate   FLOAT,
  engagement    FLOAT,              -- opens / clicks / replies
  age_days      INT,                -- how long we have seen this entity
  first_seen    TIMESTAMP,
  last_updated  TIMESTAMP,
  reputation     FLOAT,              -- bounded [-1.0 .. +1.0]
  PRIMARY KEY (entity_type, entity_id),
  INDEX idx_rep  (reputation),
  INDEX idx_seen (last_updated)
);
```

### Score Update Rule

- **Bayesian smoothing:**  $rep = (spam + \alpha \cdot m) / (total + \alpha)$ ,  $m$  = global prior,  $\alpha$  = smoothing strength
- **Decay:** 30-day exponential half-life so old behaviour fades
- **Cold-start penalty:** entities with age < 24 h get a damped score (treat as 'unknown'  $\approx$  slight negative)
- **Three tiers blended:**  $0.5 \times IP + 0.3 \times domain + 0.2 \times ASN$ , then clipped to  $[-1, +1]$
- **Update cadence:** hot increments on each verdict (Redis INCRBY); full recompute hourly in Spark

### Hot Path Lookup

```
# Pre-filter looks up all three keys in a single Redis MGET
ip, dom, asn = parse_envelope(msg)
rep_ip, rep_dom, rep_asn = redis.mget(
```

```
    f'rep:IP:{ip}', f'rep:DOMAIN:{dom}', f'rep:ASN:{asn}'
)
reputation = (
    0.5 * float(rep_ip or 0.0) +
    0.3 * float(rep_dom or 0.0) +
    0.2 * float(rep_asn or 0.0)
)
if reputation < -0.8:
    return DROP # known bad sender, never reaches model
if reputation > 0.6:
    bypass_l4 = True # high-reputation senders skip BERT
```

#### TIP

##### ASN matters

Snowshoe spammers rotate IPs cheaply but ASNs are economically expensive to change. A bad ASN reputation is one of the most stable adversarial signals available.

# 07

## Content Fingerprinting

SimHash + LSH for near-duplicate spam waves

Spam campaigns blast the same payload (with cosmetic mutations) to millions of recipients in a short window. Detecting **near-duplicate** bodies in  $O(1)$  is therefore one of the highest-leverage signals in the pipeline. We use 64-bit **SimHash** over weighted shingles, indexed by **Locality-Sensitive Hashing**.

### SimHash + LSH (Algorithm)

```
# 1) Shingle the normalised body into k-grams (k=4 words)
shingles = ngrams(normalize(body), n=4)

# 2) SimHash: weighted sum of shingle-hash bits
v = [0]*64
for sh in shingles:
    h = murmur64(sh)          # 64-bit hash
    w = idf(sh)               # rare shingles weigh more
    for i in range(64):
        v[i] += w if (h >> i) & 1 else -w
fp = 0
for i in range(64):
    if v[i] > 0: fp |= (1 << i)

# 3) LSH: split fingerprint into 4 × 16-bit bands.
#     Two messages within Hamming-3 differ in at most one band
#     (pigeonhole), so candidates are found in  $O(1)$ .
bands = [(fp >> (16*i)) & 0xFFFF for i in range(4)]
for i, b in enumerate(bands):
    candidates |= redis.smembers(f'lsh:{i}:{b}')

# 4) Verify Hamming distance on the candidate set
near_dups = [c for c in candidates if popcount(fp ^ c) <= 3]

# 5) Index for future probes (TTL = 24h sliding window)
for i, b in enumerate(bands):
    redis.sadd(f'lsh:{i}:{b}', fp)
    redis.expire(f'lsh:{i}:{b}', 86400)
```

### Why It Works

- **Robust to small mutations:** reordered words, swapped synonyms, padded whitespace
- **Cheap:**  $O(\text{shingles})$  hashing per message; index probe is  $O(1)$  Redis set lookup
- **Wave detection:** if a fingerprint appears  $> N$  times in 5 minutes across  $> M$  senders → instant block

- **Bounded recall by design:** LSH with  $b$  bands,  $r$  bits/band approximates Hamming threshold via  $1 - (1 - s^r)^b$
- **Privacy:** we only ever store 64-bit fingerprints, never plaintext

#### INSIGHT

##### MinHash is not the right tool here

MinHash estimates Jaccard on sets — great for document retrieval. SimHash measures cosine on weighted vectors and is more robust to small edits, which is exactly the spammer's tactic. Pick the hash that matches the adversary's edit budget.

# 08

## Feature Extraction

What goes into the model

### Feature Vector Layout

```
# Wire format consumed by L3 (LightGBM) and L4 (BERT)
FeatureVector {
    // ----- envelope (numeric / categorical) -----
    ip_reputation      : float32  // [-1.0 .. 1.0]
    domain_reputation  : float32
    asn_reputation     : float32
    domain_age_days    : int32
    spf_pass           : bool
    dkim_pass          : bool
    dmarc_pass         : bool
    tls_used           : bool

    // ----- behavioural -----
    msgs_from_ip_5m    : int32    // velocity
    unique_recips_5m   : int32    // fan-out
    avg_recip_open_rate : float32  // 7d

    // ----- content -----
    body_len_bytes     : int32
    link_count         : int16
    shortener_link_count : int16
    attachment_count   : int8
    image_to_text_ratio : float32  // OCR-derived
    language           : enum32
    simhash            : uint64
    near_dup_count_24h : int32    // LSH probe result

    // ----- TF-IDF (sparse, for L3 baseline) -----
    tfidf_topk         : list<uint32 token_id, float32 weight> // top-128

    // ----- text (for L4 only when borderline) -----
    body_tokens        : list<int32> // BPE ids, max 512
}
```

### Counter-Adversarial Features

- **OCR on images:** spammers hide text inside JPEGs to dodge TF-IDF — Tesseract / on-device OCR adds it back
- **URL shortener resolution:** follow bit.ly / t.co at extraction time; classify the destination, not the disguise

- **Server-side render:** for HTML mail, render with headless Chromium and re-extract visible text
- **Punycode / homoglyph normalisation:** apple.com (Cyrillic a) becomes apple.com
- **Zero-width / RTL-override stripping:** remove invisible Unicode that confuses tokenisers

## Online Feature Store

---

- **Velocity counters** (msgs\_from\_ip\_5m, unique\_recips\_5m) live in Redis Streams + sliding-window structures
- **Reputation** hits the Redis layer described in §06
- **Recipient-side features** (avg\_recip\_open\_rate) are joined from Cassandra by user-id
- **Single round trip:** Feature Extractor issues one MGET / pipelined call < 5 ms p99

# 09

## Model Stack

Naive Bayes → LightGBM → BERT

We deliberately ship a portfolio of models with very different complexity / latency / accuracy trade-offs. The portfolio also limits blast radius: if BERT regresses we can route 100% to LightGBM in seconds.

### Model Comparison

| Model                 | Latency       | AUC (typical) | Strength                                       | Weakness                                         |
|-----------------------|---------------|---------------|------------------------------------------------|--------------------------------------------------|
| TF-IDF + Naive Bayes  | ~ 100 $\mu$ s | 0.92          | Trivial to train & explain; great baseline     | Brittle to obfuscation; ignores envelope         |
| LightGBM (GBDT)       | ~ 1 ms        | 0.97          | Mixes dense & sparse features well; calibrated | Doesn't read raw text; needs feature engineering |
| Fine-tuned BERT-small | ~ 8 ms (T4)   | 0.985         | Reads body; catches novel phishing wording     | Costly; needs GPUs; harder to explain            |
| DistilBERT distilled  | ~ 3 ms        | 0.98          | BERT quality at GBDT-ish cost                  | Still needs GPUs; complex deploy                 |

### Routing Logic

```
score_gbdtd = lightgbm.predict(features)
if score_gbdtd < 0.05:      return ALLOW
if score_gbdtd > 0.95:      return SPAM
if reputation < -0.5:        return SPAM      # cheap precision boost
if 0.4 <= score_gbdtd <= 0.7: # borderline → call deep model
    score_bert = bert.predict(features.body_tokens)
    final = 0.4 * score_gbdtd + 0.6 * score_bert
    return SPAM if final > 0.6 else ALLOW
return SPAM if score_gbdtd > 0.5 else ALLOW
```

### Why a Per-Message Model and Not Just Per-Sender

| Granularity | Pros                       | Cons                                                                |
|-------------|----------------------------|---------------------------------------------------------------------|
| Per-sender  | Cheap; one decision per IP | Punishes shared infrastructure (Gmail, Outlook IPs); zero-day blind |



| Granularity   | Pros                                                | Cons                                                   |
|---------------|-----------------------------------------------------|--------------------------------------------------------|
| Per-message   | Catches novel content even from clean senders       | Higher compute; per-row latency budget                 |
| Hybrid (ours) | Sender reputation gates first, content model second | More moving parts; needs careful threshold calibration |

#### INSIGHT

##### Champion / Challenger

Run two models simultaneously: **champion** serves traffic, **challenger** shadow-scores. Promote the challenger only after 7 days of better precision-at-fixed-recall on a held-out, daily-refreshed slice — never on training-set metrics.

# 10

## Decision & Audit Log

Every verdict is explainable

Every message that reaches L1 produces an audit row. The row is the single source of truth for analytics, appeals, training labels and regulatory disclosure. We persist it for 90 days hot (BigQuery / Snowflake) and 7 years cold (S3 Glacier with hashed message-ids only).

### Spam Decision Audit Schema

```
CREATE TABLE spam_decision (  
  message_id      UUID PRIMARY KEY,  
  tenant_id       BIGINT,  
  ts              TIMESTAMP,  
  envelope_from   VARCHAR(254),  
  ip              INET,  
  asn             INT,  
  simhash         BIGINT,  
  ip_reputation   FLOAT,  
  domain_reputation FLOAT,  
  rule_hits       TEXT[],          -- which L2 rules fired  
  score_gbdtdt    FLOAT,  
  score_bert      FLOAT,          -- nullable; only on borderline  
  final_score     FLOAT,  
  verdict         ENUM('ALLOW', 'SPAM', 'QUARANTINE', 'DROP'),  
  reason_code     VARCHAR(32),    -- 'L0_RBL', 'L3_HIGH', 'L4_BORDER'...  
  model_version   VARCHAR(32),  
  feedback_label  ENUM('NONE', 'SPAM', 'HAM') DEFAULT 'NONE',  
  feedback_ts     TIMESTAMP NULL,  
  INDEX idx_tenant_ts (tenant_id, ts),  
  INDEX idx_verdict_ts (verdict, ts),  
  INDEX idx_ip_ts   (ip, ts)  
) PARTITION BY RANGE (ts); -- daily partitions
```

### Reason Codes

- **L0\_RBL**: sender IP on real-time blacklist; rejected at SMTP HELO
- **L1\_AUTH\_FAIL**: SPF / DKIM failed AND DMARC = reject
- **L2\_RULE\_<id>**: a specific operator-defined rule fired
- **L3\_HIGH / L3\_LOW**: GBDT confident enough alone
- **L4\_BORDER**: blended GBDT + BERT score crossed threshold
- **FINGERPRINT\_WAVE**: SimHash matched > 1 K messages in last 5 min

#### TIP

##### **Reason codes are a product feature**

When a user opens an appeals ticket, the support agent reads the reason code, not the raw score. Make the codes few, stable and human-readable.

# 11

## Feedback Loop

Closing the learning circle

The model is only as good as its labels. We harvest implicit and explicit signals continuously and close the loop nightly.

### Label Sources

| Signal                    | Quality         | Volume        | Latency |
|---------------------------|-----------------|---------------|---------|
| User 'Report spam'        | High (positive) | Millions/day  | Seconds |
| User 'Not spam'           | High (negative) | 100 K/day     | Seconds |
| Honeypot inboxes          | Very high       | Tens of K/day | Seconds |
| Bounce / NDR              | Medium          | Billions      | Minutes |
| Engagement (open / reply) | Weak negative   | Hundreds of B | Minutes |
| Hand-curated ground truth | Gold            | Thousands     | Days    |

### Pipeline

1. User clicks *Report spam* → client emits event to Kafka (spam\_feedback) within seconds
2. Stream processor joins on message\_id with the existing spam\_decision row
3. Hot loop: increment per-IP / per-domain spam counters in Redis (effective in < 60 s)
4. Warm loop: write label to a daily Parquet partition in S3
5. Nightly Spark job builds the training set, weights labels by source quality, holds out 7-day rolling slice
6. Train challenger (LightGBM + DistilBERT) on Ray cluster, push artefact to model registry
7. Model server canaries the challenger to 1% of traffic, ramps to 10% / 50% / 100% over 24 h gated on regressions

#### WARNING

##### Beware label leakage

Users only mark messages they actually saw — i.e., that we already let through. Train naïvely on these and the model collapses to 'predict your own past decisions'. Counter with: honeypot accounts, stratified sampling across verdict buckets, and counter-factual reweighting by inverse propensity ( $1 / P(\text{seen})$ ) to undo the selection bias.

# 12

## Adversarial ML

The arms race never ends

Spammers have economic incentive to probe. Treat the system like a security product, not a one-shot ML model.

### Common Evasions and Counter-Measures

| Evasion                       | How it works                         | Counter-measure                                             |
|-------------------------------|--------------------------------------|-------------------------------------------------------------|
| Image-only spam               | Body is a single JPEG of text        | Tesseract OCR; treat OCR text as body input                 |
| URL shortener chain           | bit.ly → t.co → goo.gl → bad.example | Resolve at extraction time, score the destination           |
| Homoglyph domains             | apple.com with Cyrillic 'a'          | Punycode normalise; flag mixed-script domains               |
| Zero-width chars              | free money                           | Strip C0 / RTL / ZW characters before tokenising            |
| Snowshoe (low-volume per IP)  | Spread across 10 K cheap IPs         | ASN reputation; per-tenant fan-out features                 |
| Hashbusting                   | Append random words to break SimHash | Weight shingles by IDF; ignore long random tails            |
| Reply-chain hijack            | Inject reply into legit thread       | Verify In-Reply-To DKIM continuity                          |
| Adversarial text perturbation | Replace 'viagra' → 'v1agra'          | Character-level model; subword BPE; OCR-style normalisation |
| Compromised legit account     | Real Gmail account turns bad         | Behavioural anomaly: sudden fan-out / unusual hours         |

### Defensive Posture

- **Rate-limit feedback:** attackers submit fake 'not spam' reports — cap weight per account & require account age
- **Don't leak scores:** never expose raw model probabilities to senders; round verdicts coarsely
- **Diversity of models:** ensembles are harder to gradient-attack than a single deep net
- **Honeypots:** seeded inboxes that should never receive mail; anything landing there is gold-label spam
- **Shadow traffic:** tee 1% of inputs to a paranoid challenger model to detect drift early

# 13

## Scalability & Distribution

Sharding, replication, multi-region

### Horizontal Scaling

---

- **Pre-Filter / Feature Extractor**: stateless, scaled by QPS
- **Model Server (LightGBM)**: stateless, model loaded at startup; rolling restart on new model
- **Model Server (BERT GPU)**: autoscaled by GPU utilisation; right-sized to ~10% of LightGBM QPS
- **Reputation Redis**: sharded by entity-id hash; 16 shards × 3 replicas
- **RocksDB warm tier**: sharded same scheme; daily Spark recompute writes new SSTables

### Multi-Region Topology

---

- Inline scoring is **region-local**: never wait on a cross-region call
- Reputation is replicated **active-active** via CRDT counters (PN-counters per IP)
- Model registry is **global, single-writer**: training cluster pushes artefact, regions pull on rollout
- Verdict log is **region-local first**, async-replicated to a global lake for training
- Privacy: bodies never leave region; only 64-bit fingerprints + features cross

### Capacity Math Recap

---

- L0 hosts:  $3.5 \text{ M/sec} \div 50 \text{ K/sec/host} \approx \mathbf{70 \text{ pre-filter hosts}}$
- Feature extractor:  $350 \text{ K/sec} \div 5 \text{ K/sec/host} \approx \mathbf{70 \text{ hosts}}$
- LightGBM serving:  $30 \text{ K/sec}$ ,  $\sim 1 \text{ ms/req} \approx \mathbf{30 \text{ cores} + 5 \times \text{HA} = 150 \text{ cores}}$
- BERT serving:  $3 \text{ K/sec}$ ,  $8 \text{ ms}$  on T4  $\approx \sim \mathbf{30 \text{ T4 GPUs}}$
- Reputation Redis:  $200 \text{ K reads/sec} \rightarrow$  comfortable on 16-node cluster

# 14

## Failure Modes & Recovery

What can go wrong

| Failure                          | Impact                               | Detection                   | Mitigation                                                 |
|----------------------------------|--------------------------------------|-----------------------------|------------------------------------------------------------|
| Reputation Redis down            | Cold-start everyone; precision drops | Latency / error spike       | Fail-open to neutral score; serve from RocksDB warm tier   |
| BERT GPU outage                  | Borderline cases mis-routed          | GPU health check            | Fall back to LightGBM with widened threshold; alert        |
| Model artefact corrupted         | All scores become 0.5                | Score histogram drift alarm | Auto-rollback to previous model in registry                |
| Kafka outage (verdicts)          | Audit + training data lost           | Broker dead alert           | Local disk buffer + replay; never block the inline path    |
| LSH index lag                    | Wave detection delayed               | Wave-detect SLO breach      | Increase shingle weight; secondary in-process Bloom filter |
| Feedback flood (attack)          | Skewed labels poison training        | Anomalous /spam clicks/sec  | Cap per-account weight; require account age + 2FA          |
| Training pipeline silent failure | Stale model for days                 | Model-age SLO > 24 h        | Page on-call; freeze rules in place; promote last-good     |
| Threshold mis-calibration        | FP rate > 0.1%                       | Real-time FP probe          | Auto-revert thresholds; canary halts further rollout       |

### Disaster Recovery

- **RTO:** < 5 min to fail-open per region; < 30 min to redeploy a good model
- **RPO:** < 1 min for verdict log; never block delivery on storage failure
- **Backups:** model registry replicated 3× regions; warm reputation snapshot daily
- **Game days:** quarterly chaos drill — kill BERT pool, kill Redis primary, replay yesterday's spam wave

#### WARNING

##### Fail-open, not fail-closed

If the spam pipeline crashes, it is far better to deliver a few extra spammy messages than to drop legitimate mail. Default to ALLOW with a low-confidence flag and rely on layer-0 (RBL) to keep the floor.

# 15

## Design Trade-offs

Decisions and rationale

| Decision                   | Choice                                       | Trade-off                                                                              |
|----------------------------|----------------------------------------------|----------------------------------------------------------------------------------------|
| FP vs FN                   | Asymmetric: FP < 0.1% is sacred              | A legit email in spam is far worse than a spam in inbox; tune thresholds accordingly   |
| Latency vs accuracy        | Cascade with 50 ms p99 budget                | Cheap layers get most decisions; deep model only for borderline (~0.1% of traffic)     |
| Rules vs ML                | Hybrid                                       | Rules ship in hours but rot fast; ML adapts but has cold-start; we keep both           |
| Per-msg vs per-sender      | Hybrid                                       | Reputation gates first (cheap), content model decides borderline                       |
| Deep model vs GBDT         | GBDT champion, BERT challenger on borderline | GBDT is faster, more explainable; BERT shines on novel phishing wording                |
| Sync vs async scoring      | Sync inline                                  | Email cannot be 'un-delivered' cheaply; spam folder requires the verdict at write time |
| Global vs per-tenant model | Global base + per-tenant rule overlays       | Captures shared patterns, lets enterprises encode their own policy                     |
| Storage of bodies          | Never persisted across borders               | Strong privacy posture; loses some debug power                                         |
| Threshold tuning           | Per-tenant ROC-curve picker, monthly         | Some tenants accept higher FN for lower FP (consumer) and vice-versa (B2B alerts)      |

### KEY

#### Asymmetric error costs

Internal SLO: 1 false-positive = 100 false-negatives in cost. The pipeline is engineered around that ratio — every threshold, every routing rule, every model promotion gate is calibrated for it.



# 16

## Interview Playbook

How to present this

Spam detection is one of the few interview questions where **ML** and **systems** meet at full force. Most candidates over-index on the model and forget the 3.5 M/sec constraint. Lead with the cascade.

### 45-Minute Outline

---

1. **Intro (2 min)**: clarify volume, latency, FP/FN asymmetry, adversarial setting
2. **Capacity (3 min)**: 300 B/day → 3.5 M/sec; p99 50 ms; explain why a single big model is impossible
3. **Layered defence (8 min)**: draw the 5-layer funnel; argue 10× rejection per layer
4. **Reputation (5 min)**: per-IP / per-domain / per-ASN; Bayesian smoothing; ASN as anti-snowshoe lever
5. **Fingerprinting (5 min)**: SimHash + LSH; explain why MinHash is the wrong tool
6. **Model stack (6 min)**: NB → LightGBM → BERT; routing rule with explicit thresholds
7. **Feedback loop (5 min)**: Kafka → Spark → registry; mention selection bias and propensity weighting
8. **Adversarial (5 min)**: name 3 concrete evasions and counter-measures
9. **Failures & trade-offs (4 min)**: fail-open; FP < FN cost; champion / challenger
10. **Wrap (2 min)**: what you would build first, what you would defer

### Key Talking Points

---

- “3.5 M/sec at 50 ms p99 — that’s why a cascade exists, not because we love complexity”
- “Each layer rejects an order of magnitude; BERT only sees 0.1% of inputs”
- “Reputation at three granularities — IP, domain, ASN — to defeat snowshoe spam”
- “SimHash + LSH catches near-duplicate waves in  $O(1)$ ”
- “FP costs 100× FN; the entire pipeline is calibrated to that ratio”
- “Champion / challenger with shadow traffic, never trust offline metrics alone”
- “Fail-open — never delay legitimate email because the ML stack hiccupped”
- “Selection bias is the silent killer of every spam model”

### Common Follow-ups & Answers

---

- **Q: Why not one big neural net?** A:  $3.5 \text{ M/sec} \times 8 \text{ ms BERT} = 28 \text{ K GPU-seconds/sec} \approx 28 \text{ K T4s}$ . Cascade trims that by 1000×.
- **Q: How do you bootstrap a brand-new sender?** A: ASN reputation + domain-age penalty + content model. Cold senders get a damped neutral score, not a free pass.
- **Q: How do you avoid label leakage?** A: Honeypots + stratified sampling across verdict buckets + inverse-propensity weighting in the loss.

- **Q: What if a spammer flips a model with adversarial perturbations?** A: Ensemble of NB + GBDT + BERT; subword tokenisation; never expose raw scores.
- **Q: How fast can you react to a new wave?** A: SimHash detects within minutes; rules ship within hours; full retrain nightly.
- **Q: How do you handle 'this is spam' vs 'this is not spam' class imbalance?** A: Class-weighted loss + focal loss for BERT; precision-at-fixed-recall as the headline metric.

#### KEY

##### Memorize These Numbers

3.5 M/sec · p99 < 50 ms · 5 layers, 10× reject each · FP < 0.1% · FN < 1% · FP cost = 100 × FN cost  
· BERT sees 0.1% of traffic · Nightly retrain, 1% canary.